

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC NGOẠI THƯƠNG



BÁO CÁO TIỂU LUẬN VÀ THỰC HÀNH
MÔN HỌC
MẠNG MÁY TÍNH

ĐỀ TÀI:

**PHÂN TÍCH VÀ PHÁT HIỆN CÁC KHOẢNG THỜI GIAN MẠNG
SUY GIẢM CHẤT LƯỢNG TỪ DỮ LIỆU ĐO ĐỊNH KỲ**

Sinh viên thực hiện	: Bùi Anh Đức
Giảng viên	: TS Ngô Hải Anh
Khoa	: Công nghệ và khoa học dữ liệu
Lớp	: COSH201 (2627.1 - GD1).1

Hà Nội - 2026

PHIẾU CHẤM ĐIỂM

Sinh viên thực hiện:

Họ và tên	Chữ ký	Ghi chú
Bùi Anh Đức		

Giảng viên chấm:

Họ và tên	Điểm	Chữ ký
Ngô Hải Anh		

MỤC LỤC

DANH MỤC CHỮ VIẾT TẮT	5
DANH MỤC BẢNG BIỂU	6
CHƯƠNG 1: GIỚI THIỆU	6
1.1 Bối cảnh và vấn đề nghiên cứu.....	7
1.2 Mục tiêu và câu hỏi nghiên cứu	7
1.3 Phạm vi nghiên cứu.....	8
1.4 Phương pháp và kết quả mong đợi.....	8
1.5 Ý nghĩa của kết quả dự kiến.....	9
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	10
2.1 Những thành phần mạng liên quan đến phép đo.....	10
2.2 RTT và độ dao động RTT	10
2.3 Tỷ lệ không nhận phản hồi ICMP	12
2.4 DNS latency.....	12
2.5 HTTP latency và các thành phần thời gian	13
2.6 Dữ liệu theo thời gian và cửa sổ trượt.....	14
2.7 Isolation Forest và ngưỡng đối chiếu	15
2.8 Nguyên tắc đánh giá và diễn giải	16
CHƯƠNG 3: DATASET VÀ PHƯƠNG PHÁP.....	18
3.1 Nguồn dữ liệu và đối chiếu log.....	18
3.2 Đơn vị quan sát và cách lấy mẫu	18
3.3 Data dictionary	20
3.4 Làm sạch và kiểm tra chất lượng	22
3.5 Tạo đặc trưng theo thời gian.....	24
3.6 Thiết kế đánh giá Isolation Forest.....	26
CHƯƠNG 4: XÂY DỰNG CHƯƠNG TRÌNH VÀ THỰC NGHIỆM.....	27
4.1 Kiến trúc chương trình	27

4.2 Thu thập định kỳ và ghi nhật ký	27
4.3 Thực hiện bước làm sạch.....	28
4.4 Tính cửa sổ và đặc trưng	29
4.5 Huấn luyện Isolation Forest	30
4.6 Ngưỡng đơn giản dùng để đối chiếu.....	31
4.7 Gộp thành khoảng thời gian và giải thích chỉ số.....	32
4.8 Phân tích bổ sung các thành phần HTTP	33
4.9 Cách tái lập và kiểm tra.....	34
CHƯƠNG 5: KẾT QUẢ VÀ THẢO LUẬN.....	35
5.1 Timeline cảnh báo Isolation Forest	35
5.2 Điểm bất thường và các khoảng được phát hiện.....	36
5.3 Giải thích cảnh báo và trường hợp gần ngưỡng	36
5.4 Quan hệ RTT DNS và HTTP latency	37
5.5 Phân tích nâng cao khi HTTP tăng cao	37
5.6 Baseline đối chiếu và giới hạn	40
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	41
6.1 Kết luận	41
6.2 Hướng phát triển.....	41
TÀI LIỆU THAM KHẢO	42
PHỤ LỤC	44

DANH MỤC CHỮ VIẾT TẮT

Viết tắt	Tên đầy đủ	Ý nghĩa và cách sử dụng trong bài
RTT	Round-Trip Time	Thời gian khứ hồi của gói tin, dùng để đánh giá độ trễ phản hồi.
DNS	Domain Name System	Hệ thống phân giải tên miền thành địa chỉ IP; nhóm đo độ trễ phân giải.
HTTP / HTTPS	Hypertext Transfer Protocol / Hypertext Transfer Protocol Secure	Giao thức trao đổi web; HTTPS được bảo vệ bằng TLS. Nhóm đo thời gian thực hiện yêu cầu web.
ICMP	Internet Control Message Protocol	Giao thức được ping sử dụng để kiểm tra phản hồi và tính các chỉ số mạng.
IF	Isolation Forest	Mô hình phát hiện mẫu khác thường, hỗ trợ xác định khoảng mạng có dấu hiệu suy giảm.
P95	95th Percentile	Phân vị thứ 95, dùng để mô tả mức độ trễ cao trong cửa sổ đo.
IQR	Interquartile Range	Khoảng tứ phân vị, bằng P75 – P25; dùng để chuẩn hóa mức tăng của các chỉ số.
TLS	Transport Layer Security	Giao thức bảo vệ kết nối HTTPS; thời gian thiết lập TLS hỗ trợ giải thích HTTP chậm.

DANH MỤC BẢNG BIỂU

<i>Bảng 1. Thông tin dữ liệu thu thập.....</i>	19
<i>Bảng 2. Từ điển dữ liệu</i>	20
<i>Bảng 3. Giá trị thiếu của năm chỉ số chính.....</i>	22
<i>Bảng 4. Chín đặc trưng đầu vào mô hình</i>	25
<i>Bảng 5. Các thành phần của bộ chương trình</i>	27
<i>Bảng 6. Số lượng và tỷ lệ cảnh báo trên tập kiểm tra.....</i>	31
<i>Bảng 7. Ngưỡng baseline trong thực nghiệm</i>	32
<i>Bảng 8. Bốn khoảng IF dài nhất trên test</i>	36
<i>Bảng 9. Đối chiếu số đo góc của các thành phần trong khoảng IF ngày 02/09 39</i>	
<i>Bảng 10. Đối ngưỡng từ điểm train của cùng rừng đã huấn luyện</i>	45
<i>Bảng 11. Thống kê các giá trị hợp lệ trước khi tạo cửa sổ.....</i>	47
 <i>Hình 1. Cảnh báo IF trên năm chỉ số trung bình trượt ở tập kiểm tra; vùng xám chưa có dữ liệu cửa sổ.</i>	35
<i>Hình 2. Tương quan Pearson giữa RTT, DNS và HTTP trên các cặp giá trị hợp lệ.</i>	37
<i>Hình 3. Pearson và Spearman của TCP, TLS, TTFB và HTTP trên cùng các lượt đủ dữ liệu.....</i>	38
<i>Hình 4. Phân bố điểm IF của train và test, với ngưỡng 0,601695 từ train.</i>	45
<i>Hình 5. Tỷ lệ cảnh báo theo ngày, Isolation Forest là cột chính và ngưỡng là cột đối chiếu.</i>	46
<i>Hình 6. Trung bình RTT và HTTP theo những giờ có dữ liệu.</i>	46
<i>Hình 7. Tỷ trọng TCP, TLS, TTFB và phần đọc còn lại trong ba trường hợp. ..</i>	48

CHƯƠNG 1: GIỚI THIỆU

1.1 Bối cảnh và vấn đề nghiên cứu

Khi học trực tuyến, tìm kiếm tài liệu hoặc sử dụng các trang web, người dùng có thể gặp tình trạng mạng chậm trong một khoảng thời gian rồi tự trở lại bình thường. Một lần kiểm tra kết nối chỉ cho biết tình trạng tại thời điểm kiểm tra. Nếu sự cố xảy ra trước đó, người dùng khó trả lời được mạng bắt đầu chậm lúc nào, kéo dài bao lâu và biểu hiện nào thay đổi rõ nhất. Vì vậy, cần ghi lại các phép đo theo thời gian để có căn cứ phân tích.

Chất lượng kết nối không thể được mô tả đầy đủ bằng một con số. Thời gian phản hồi của gói kiểm tra, mức dao động của thời gian phản hồi, khả năng nhận phản hồi, thời gian phân giải tên miền và thời gian thực hiện yêu cầu web phản ánh các khía cạnh khác nhau. Một máy tính có thể vẫn nhận phản hồi ping nhưng việc truy cập web lại chậm. Ngược lại, ping không nhận được phản hồi chưa đủ để khẳng định tất cả dịch vụ đều ngừng hoạt động.

Từ vấn đề này, nhóm em thực hiện đề tài T19: Phân tích và phát hiện các khoảng thời gian mạng suy giảm chất lượng từ dữ liệu đo định kỳ. Nhóm tập trung vào đọc hiểu số liệu mạng và xác định những khoảng cần kiểm tra. Nhóm sử dụng Machine Learning như một công cụ hỗ trợ tìm các mẫu khác thường trong dữ liệu.

1.2 Mục tiêu và câu hỏi nghiên cứu

Mục tiêu của nhóm em là xây dựng một chương trình Python nhỏ, có thể chạy lại trên bộ dữ liệu cập nhật được cung cấp, để mô tả chất lượng kết nối theo thời gian và phát hiện các khoảng có dấu hiệu suy giảm. Chương trình cần xuất được biểu đồ, danh sách khoảng bất thường và các chỉ số giúp giải thích kết quả.

Nhóm tập trung vào ba câu hỏi:

1. Các chỉ số chất lượng kết nối biến động như thế nào và những khoảng thời gian nào đáng chú ý?

2. Isolation Forest phát hiện các mẫu khác thường nào và quy tắc ngưỡng giúp đối chiếu kết quả ra sao?

3. Trong các khoảng được phát hiện, chỉ số nào tăng rõ nhất và RTT, DNS latency, HTTP latency có cùng biến động hay không?

Để trả lời các câu hỏi, nhóm em thực hiện lần lượt việc kiểm tra dữ liệu, tính các đặc trưng theo cửa sổ thời gian, áp dụng Isolation Forest và đối chiếu với quy tắc ngưỡng và xem lại các trường hợp cụ thể. Nhóm giải thích kết quả bằng ý nghĩa của phép đo mạng, không chỉ dựa vào nhãn do chương trình tạo ra.

1.3 Phạm vi nghiên cứu

Đây là tiểu luận do nhóm em thực hiện. Nhóm dùng phương pháp chính là Isolation Forest, dùng các chỉ số đo theo thời gian để tìm những mẫu khác thường. Baseline theo ngưỡng chỉ là phương pháp đối chiếu để giải thích. Nhóm không dùng baseline làm nhãn đúng để huấn luyện hoặc chấm độ chính xác của mô hình.

Nhóm thu thập dữ liệu trên máy tính để bàn hệ điều hành Windows, kết nối Ethernet. Collector có cấu hình ping 8.8.8.8, phân giải www.google.com và thực hiện yêu cầu HTTPS tới www.google.com. Dữ liệu nhóm sử dụng là file CSV gồm 51.070 dòng. Nguồn file và các điểm cần đối chiếu được trình bày ở Chương 3.

Nhóm chỉ phân tích các phép đo và thời điểm thực sự có trong file. Nhóm không đo thông lượng tối đa, không xác định router gây lỗi và không dùng khoảng trống trong CSV để kết luận mất mạng.

1.4 Phương pháp và kết quả mong đợi

Nhóm thực hiện sáu bước: đọc dữ liệu; làm sạch; phân tích thống kê; tạo đặc trưng; áp dụng Isolation Forest, đối chiếu baseline; đánh giá và trực quan hóa. Nhóm giữ nguyên thứ tự thời gian khi thiết kế thực nghiệm. Nhóm không dùng thông tin của giai đoạn đánh giá để học cách điền thiếu, chọn ngưỡng thống kê

hoặc huấn luyện mô hình. Nguyên tắc này giúp tránh sử dụng thông tin tương lai để đánh giá dữ liệu quá khứ. [\[10\]](#)

Sản phẩm của nhóm gồm mã Python, dataset thực sự được sử dụng, README hướng dẫn chạy, các biểu đồ có diễn giải và báo cáo sáu chương. Kết quả phát hiện có vai trò gợi ý những khoảng cần xem xét. Để kiểm tra cảnh báo có trùng với sự cố thực tế hay không, cần đối chiếu thêm với ghi chép của người sử dụng hoặc một nguồn giám sát khác.

1.5 Ý nghĩa của kết quả dự kiến

Nhóm hướng tới trả lời những câu hỏi như khoảng nào nên xem lại, chỉ số nào tăng rõ và một cảnh báo có kéo dài qua nhiều lượt đo hay không. Nhóm tập trung giải thích điểm bất thường và các khoảng do Isolation Forest đánh dấu. Kết quả theo ngưỡng giúp người đọc có thêm mốc đối chiếu trực tiếp với đơn vị mili giây và phần trăm.

Một mô hình đánh dấu ít mẫu không tự động có nghĩa là chính xác hơn, cũng như nhiều cảnh báo không tự động có nghĩa là phát hiện đầy đủ sự cố. Nhóm sử dụng số cảnh báo, phân bố điểm, độ nhạy của ngưỡng và các trường hợp cụ thể để đánh giá. Nhóm chưa tính Precision, Recall và F1 khi không có nhãn sự cố đã xác nhận.

Trong bài, nhóm em tập trung làm rõ một cửa sổ thời gian, một mô hình cơ bản và một quy tắc gộp khoảng. Khi cần giải thích một kết quả, nhóm có thể truy từ biểu đồ về intervals.csv, rồi xem từng lượt trong scored_data.csv. Cách làm này gắn phân lập trình với ý nghĩa phép đo mạng thay vì chỉ báo cáo một nhãn từ thuật toán.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Những thành phần mạng liên quan đến phép đo

Quá trình truy cập một dịch vụ HTTPS bao gồm nhiều bước, từ phân giải tên miền, thiết lập kết nối đến trao đổi dữ liệu giữa máy khách và máy chủ. Trong chương trình thu thập của đề tài, yêu cầu web sử dụng HTTP/1.1 trên kết nối TCP được bảo vệ bằng TLS. TCP cung cấp cơ chế truyền dữ liệu có thứ tự và truyền lại khi cần thiết, góp phần bảo đảm tính tin cậy của quá trình trao đổi. [5] HTTP quy định cách trao đổi yêu cầu và phản hồi ở tầng ứng dụng, trong khi HTTPS bảo vệ hoạt động trao đổi này thông qua TLS. [6]

Để quan sát chất lượng kết nối, nhóm kết hợp ba phép đo: ping, phân giải DNS và truy cập HTTPS. Ping sử dụng bản tin ICMP Echo Request và Echo Reply để kiểm tra khả năng phản hồi của một địa chỉ đích. [1] DNS thực hiện việc tra cứu thông tin tên miền, hỗ trợ xác định địa chỉ IP cần kết nối. [4] Phép đo HTTPS phản ánh thời gian thực hiện yêu cầu tới dịch vụ web. Ba phép đo cung cấp những thông tin bổ sung cho nhau, giúp đánh giá biểu hiện của kết nối ở các bước khác nhau.

Trong cấu hình nhóm sử dụng, ping được gửi tới địa chỉ 8.8.8.8, còn yêu cầu HTTPS được gửi tới máy chủ có địa chỉ IP được phân giải từ tên miền www.google.com. Vì vậy, các phép đo không hoàn toàn đại diện cho cùng một đích hoặc cùng một dịch vụ. Nhóm sử dụng dữ liệu thu được để phân tích những dấu hiệu suy giảm tại máy đo, nhưng chưa đủ để xác định chính xác thiết bị mạng, nhà cung cấp hoặc máy chủ gây ra hiện tượng.

2.2 RTT và độ dao động RTT

RTT, viết tắt của *Round-Trip Time*, là khoảng thời gian tính từ khi gửi một gói tin đến khi nhận được phản hồi tương ứng. Chỉ số này phản ánh thời gian khứ hồi của quá trình trao đổi và chịu ảnh hưởng của cả chiều gửi lẫn chiều

phản hồi. [2] RTT thường được biểu diễn bằng mili giây; giá trị tăng cho thấy máy đo phải chờ lâu hơn để nhận phản hồi.

Trong `collector.py`, mỗi lượt đo dự kiến gửi bốn gói ping. Trường `rtt_ms` được tính bằng trung bình RTT của các phản hồi mà chương trình đọc được. Khi không nhận được phản hồi nào, chương trình không có cơ sở để tính RTT và để trống giá trị tương ứng. Trường hợp này cần được phân biệt với RTT bằng 0 ms, vì giá trị thiếu không thể hiện một kết nối có độ trễ thấp.

Bên cạnh giá trị trung bình, sự biến động của độ trễ cũng là một đặc điểm cần quan tâm khi đánh giá chất lượng mạng. [3] Trong đề tài, nhóm dùng `jitter_ms` để biểu thị mức dao động giữa các RTT liên tiếp trong cùng một lượt đo. Chỉ số này được tính bằng trung bình chênh lệch tuyệt đối giữa các giá trị RTT nhận được.

Với N phản hồi hợp lệ và các giá trị RTT tương ứng là $R_1, R_2, \dots, R_N$, công thức được xác định như sau:

$$\text{jitter} = \frac{1}{N-1} \sum_{i=2}^N |R_i - R_{i-1}|, \quad N \geq 2$$

Nếu nhận đủ bốn phản hồi, công thức trở thành:

$$\text{jitter} = \frac{|R_2 - R_1| + |R_3 - R_2| + |R_4 - R_3|}{3}$$

Ví dụ minh họa, với bốn giá trị RTT lần lượt là 20, 21, 40 và 22 ms, jitter được tính bằng:

$$\text{jitter} = \frac{1 + 19 + 18}{3} \approx 12,67 \text{ ms}$$

Các giá trị trên chỉ nhằm minh họa cách tính, không phải kết quả thực nghiệm. Khi có ít hơn hai phản hồi, jitter được để trống do không đủ dữ liệu để tính chênh lệch. Theo cách định nghĩa này, jitter càng lớn thì thời gian phản hồi giữa các gói trong cùng lượt đo càng biến động.

2.3 Tỷ lệ không nhận phản hồi ICMP

Trường `loss_rate` phản ánh tỷ lệ gói ping không nhận được phản hồi theo kết quả mà chương trình thu thập được. Với số gói gửi lớn hơn 0, tỷ lệ này có thể được xác định từ số gói gửi và số phản hồi nhận được:

$$\text{loss_rate} = \frac{n_{\text{sent}} - n_{\text{received}}}{n_{\text{sent}}}$$

Trong đó, n_{sent} là số gói đã gửi và n_{received} là số phản hồi nhận được. Nếu gửi bốn gói và nhận ba phản hồi, tỷ lệ không nhận phản hồi là 0,25, tương ứng 25%. Dữ liệu trong CSV được lưu dưới dạng tỷ lệ từ 0 đến 1; khi biểu diễn theo phần trăm, giá trị được nhân với 100.

Trong mã nguồn, collector ưu tiên lấy tỷ lệ mất phản hồi từ dòng tổng kết của lệnh ping. Công thức dựa trên số đếm được sử dụng khi chương trình không đọc được tỷ lệ này. Do đó, nếu tỷ lệ trong dòng tổng kết không thống nhất với số phản hồi mà chương trình nhận diện, nhóm cần kiểm tra thêm dữ liệu thay vì tự chọn một giá trị làm kết quả đúng.

Với bốn gói trong mỗi lượt đo, chỉ một phản hồi bị thiếu cũng tương ứng với mức thay đổi 25%. Vì vậy, kết quả của một lượt đo riêng lẻ có độ phân giải tương đối thấp. Việc tổng hợp nhiều lượt đo gần nhau giúp quan sát rõ hơn mức độ lặp lại và kéo dài của hiện tượng.

Phép đo này sử dụng ICMP nên chỉ phản ánh tình trạng nhận phản hồi của các gói ping. [1] Việc không nhận được phản hồi có thể liên quan đến mất gói, phản hồi chậm, hạn chế ICMP hoặc lỗi trong quá trình gọi và đọc kết quả lệnh ping. Vì vậy, nhóm không diễn giải `loss_rate` là tỷ lệ mất của toàn bộ lưu lượng TCP hoặc UDP trên mạng.

2.4 DNS latency

DNS là hệ thống quản lý thông tin tên miền theo cấu trúc phân cấp, hỗ trợ tra cứu các bản ghi cần thiết để thiết lập kết nối. DNS cho phép lưu kết quả trong bộ nhớ đệm nhằm giảm số lần truy vấn và rút ngắn thời gian phản hồi. [4] Do đó,

thời gian phân giải có thể thay đổi tùy theo trạng thái bộ nhớ đệm và quá trình xử lý truy vấn.

Nhóm dùng `dns_latency_ms` để biểu thị thời gian thực hiện thao tác phân giải tên miền. Khi thư viện `dnspython` được cài đặt, chương trình truy vấn bản ghi A của `www.google.com` thông qua bộ phân giải 8.8.8.8. Cách thực hiện này tránh sử dụng cơ chế tra cứu qua bộ nhớ đệm DNS của hệ điều hành, nhưng bộ phân giải vẫn có thể trả lời từ bộ nhớ đệm của chính nó.

Nếu không có `dnspython`, chương trình sử dụng `socket.getaddrinfo`. Khi đó, kết quả chịu ảnh hưởng của cơ chế phân giải tên miền trên máy, bao gồm bộ nhớ đệm và cấu hình hệ thống. Nhóm cần lưu ý sự khác biệt giữa hai cách đo khi so sánh dữ liệu thu trong các môi trường khác nhau.

DNS latency tăng cho thấy thao tác phân giải diễn ra chậm hơn ở lượt đo tương ứng. Tuy nhiên, phép đo DNS và yêu cầu HTTPS được thực hiện riêng biệt. Vì vậy, DNS latency cao chưa đủ để kết luận rằng quá trình phân giải tên miền là nguyên nhân làm HTTP latency tăng trong cùng một dòng dữ liệu.

2.5 HTTP latency và các thành phần thời gian

HTTP quy định việc trao đổi yêu cầu và phản hồi giữa máy khách và máy chủ; HTTPS thực hiện hoạt động này trên kết nối được bảo vệ bằng TLS. [6] Trong đề tài, `http_latency_ms` được tính từ thời điểm trước khi tạo kết nối đến khi vòng đọc phản hồi kết thúc. Chỉ số này phản ánh thời gian của nhiều bước liên tiếp trong quá trình truy cập dịch vụ web.

Để hỗ trợ giải thích khi HTTP latency tăng, collector ghi nhận thêm ba thành phần:

- `tcp_connect_ms`: thời gian thực hiện thao tác tạo kết nối bằng `socket.create_connection`. Do hàm này nhận tên miền, khoảng thời gian đo bao gồm cả phân giải tên miền của hệ thống và thiết lập kết nối TCP.

- `tls_handshake_ms`: thời gian từ sau bước tạo kết nối TCP đến khi hoàn thành thiết lập TLS, bao gồm việc chuẩn bị ngữ cảnh bảo mật và thực hiện bắt tay TLS.
- `ttfb_ms`: thời gian từ sau bước thiết lập TLS đến khi nhận được byte phản hồi đầu tiên, bao gồm thao tác gửi yêu cầu và chờ phản hồi.

Sau khi nhận byte đầu tiên, chương trình tiếp tục đọc phần dữ liệu còn lại. Thời gian của bước này có thể được xác định bằng HTTP latency tổng trừ đi ba thành phần đã ghi nhận. Việc phân tách giúp chỉ ra bước nào đóng góp nhiều vào tổng thời gian hoặc có mức tăng đáng chú ý trong một khoảng bất thường.

Tuy nhiên, vòng đọc có thể kết thúc khi hết thời gian chờ hoặc đạt giới hạn dung lượng, nên `http_latency_ms` không bảo đảm rằng toàn bộ nội dung trang web đã được tải đầy đủ. HTTP latency còn chịu ảnh hưởng của kích thước phản hồi, thời gian xử lý phía máy chủ và quá trình truyền dữ liệu. Vì vậy, nhóm dùng chỉ số này để phân tích thời gian truy cập dịch vụ, không xem là phép đo trực tiếp tốc độ đường truyền.

2.6 Dữ liệu theo thời gian và cửa sổ trượt

Dữ liệu của đề tài là chuỗi các lượt đo được ghi nhận theo thời gian. Mỗi dòng có một timestamp để xác định thứ tự và khoảng cách giữa các quan sát. Việc sắp xếp dữ liệu theo timestamp là cơ sở để phân tích diễn biến trước, trong và sau các khoảng được cảnh báo.

Mặc dù chương trình được cấu hình đo định kỳ, khoảng cách thực tế giữa hai lượt có thể thay đổi do lịch hoạt động, thời gian thực hiện phép đo hoặc quá trình tạm dừng chương trình. Vì vậy, nhóm cần phân tích theo timestamp thực tế, đồng thời nhận diện các khoảng ngắt trước khi tổng hợp dữ liệu.

Cửa sổ trượt là tập hợp các quan sát nằm trong một khoảng thời gian gần thời điểm đang xét. Khi thời điểm phân tích thay đổi, phạm vi cửa sổ cũng dịch chuyển tương ứng. Thư viện pandas hỗ trợ cửa sổ theo số lượng dòng hoặc theo

khoảng thời gian. Với cửa sổ theo thời gian, số quan sát bên trong có thể khác nhau nếu dữ liệu không được lấy mẫu đều. [9]

Trong bài, nhóm xây dựng các đặc trưng từ cửa sổ 5 phút, chỉ sử dụng quan sát hiện tại và quá khứ. Giá trị trung bình mô tả mức độ trễ hoặc tỷ lệ mất phản hồi trong cửa sổ; độ lệch chuẩn phản ánh mức phân tán; phân vị mô tả vị trí của các giá trị trong phân bố. P95 là mốc mà khoảng 95% giá trị không vượt quá, giúp bổ sung thông tin về những lần phản hồi chậm mà giá trị trung bình có thể chưa thể hiện rõ.

Nhóm cần xem xét các thống kê cùng số lượng quan sát hợp lệ. Khi cửa sổ có quá ít dữ liệu, giá trị trung bình hoặc P95 có thể chưa ổn định. Ngoài ra, các cửa sổ liên nhau có thể chứa nhiều quan sát chung, nên các đặc trưng liên tiếp không hoàn toàn độc lập.

Nhóm dùng đặc trưng giờ trong ngày để xem xét mối liên hệ giữa thời điểm và sự xuất hiện của cảnh báo. Thông tin này hỗ trợ mô tả xu hướng theo thời gian, nhưng không đủ để giải thích nguyên nhân của hiện tượng suy giảm.

2.7 Isolation Forest và ngưỡng đối chiếu

Isolation Forest là thuật toán phát hiện bất thường dựa trên khả năng tách riêng các quan sát thông qua những phép chia ngẫu nhiên. Mỗi cây lựa chọn đặc trưng và giá trị phân chia để chia nhỏ dữ liệu. Các quan sát khác biệt thường có thể được tách riêng sau ít bước hơn so với những quan sát nằm trong vùng dữ liệu phổ biến. Kết quả từ nhiều cây được tổng hợp để đánh giá mức độ bất thường của từng mẫu. [7]

Thuật toán không yêu cầu nhãn sự cố có sẵn trong quá trình huấn luyện. Trong scikit-learn, hàm predict trả về -1 đối với mẫu được đánh dấu bất thường và 1 đối với mẫu còn lại. Tham số contamination được sử dụng để xác định ngưỡng phân loại dựa trên dữ liệu huấn luyện, không thể hiện tỷ lệ sự cố thực tế đã được xác nhận. [8]

Trong chương trình phân tích, nhóm lưu điểm bất thường theo quy ước $\text{anomaly_score} = -\text{score_samples}$, giúp việc diễn giải thống nhất theo hướng điểm càng cao thì mẫu càng khác thường. Đây là điểm đánh giá của mô hình, không phải xác suất xảy ra sự cố.

Một quan sát khác thường chưa chắc phản ánh chất lượng mạng suy giảm. Chẳng hạn, độ trễ giảm mạnh cũng có thể tạo ra một mẫu hiếm so với dữ liệu huấn luyện. Vì vậy, nhóm cần xem xét kết quả Isolation Forest cùng chiều biến động của các chỉ số, mức độ thay đổi và sự liên tiếp của các cảnh báo. Trong đề tài, nhóm dùng mô hình để hỗ trợ lựa chọn những khoảng cần phân tích thêm.

Nhóm sử dụng baseline theo ngưỡng đơn giản để đối chiếu với Isolation Forest. Một mẫu được đánh dấu khi ít nhất một chỉ số trung bình trong cửa sổ vượt ngưỡng quy định. Ưu điểm của phương pháp là dễ thực hiện và dễ xác định chỉ số làm phát sinh cảnh báo. Tuy nhiên, ngưỡng cố định có thể không phù hợp với mọi môi trường hoặc đích đo. Các ngưỡng trong bài là lựa chọn phục vụ thực nghiệm, không được xem là tiêu chuẩn chất lượng chung cho mọi kết nối Internet.

2.8 Nguyên tắc đánh giá và diễn giải

Khi chưa có nhãn sự cố đã được xác nhận độc lập, nhóm tập trung đánh giá qua số lượng và tỷ lệ cảnh báo, phân bố điểm bất thường, các khoảng được phát hiện và mức giao nhau giữa Isolation Forest với baseline. Những kết quả này giúp mô tả hành vi của hai phương pháp, nhưng chưa phản ánh trực tiếp độ chính xác phát hiện sự cố.

Precision, Recall và F1 cần có thông tin đối chiếu để xác định cảnh báo đúng, cảnh báo nhầm và sự cố bị bỏ sót. Nếu sử dụng kết quả baseline làm nhãn đúng, các chỉ số thu được chủ yếu phản ánh mức độ tương đồng với baseline. Vì vậy, nhóm không sử dụng cách đánh giá này để khẳng định hiệu quả phát hiện sự cố thực tế.

Nhóm cần diễn giải kết quả gắn với ý nghĩa của các phép đo mạng. Với mỗi khoảng được đánh dấu, nhóm cần xác định thời điểm, số lượt đo, chỉ số tăng

nổi bật và mức độ biến động. Nhóm dùng biểu đồ cùng hệ số tương quan để xem xét sự liên hệ giữa RTT, DNS, HTTP và các thành phần thời gian HTTP. Tuy nhiên, tương quan chỉ phản ánh mối liên hệ thống kê, không tự chứng minh quan hệ nguyên nhân.

Cuối cùng, do dữ liệu được thu theo chu kỳ, nhóm xác định thời điểm bắt đầu và kết thúc của một khoảng cảnh báo từ các mốc quan sát. Các mốc này hỗ trợ khoanh vùng thời gian cần kiểm tra, nhưng không đồng nghĩa với thời điểm sự cố thực tế bắt đầu hoặc kết thúc chính xác.

CHƯƠNG 3: DATASET VÀ PHƯƠNG PHÁP

3.1 Nguồn dữ liệu và đối chiếu log

Nhóm lưu bộ dữ liệu sử dụng trong đề tài tại `du_lieu/network_quality_data.csv`, gồm 51.070 bản ghi và 15 trường dữ liệu, trong khoảng từ 17:34:00 ngày 19/08/2026 đến 01:00:00 ngày 08/09/2026, theo múi giờ UTC+7. Theo thông tin về môi trường thu thập, chương trình được thực hiện trên máy tính Windows sử dụng kết nối Ethernet. Mỗi bản ghi chứa thời điểm đo, thông tin đích đo và các chỉ số chất lượng mạng.

Nhóm sử dụng dữ liệu trong file `du_lieu/network_quality_data.csv` để phân tích. Trong quá trình thu thập, collector đồng thời ghi các số đo và thông báo hoạt động vào file `network_quality_data.log`. Nhóm dùng log để xem lại quá trình chạy chương trình khi cần. Do được tạo từ cùng kết quả đo với CSV, log không phải nguồn xác nhận độc lập về độ chính xác của số đo hoặc tình trạng sự cố mạng.

Các thông báo khởi động trong phiên bản log hiện tại ghi chu kỳ 30 giây, phù hợp với cấu hình `INTERVAL_SECONDS = 30` của collector. Khoảng cách phổ biến giữa hai bản ghi liên tiếp cũng là 30 giây. Tuy nhiên, dữ liệu vẫn có các khoảng ngắt theo lịch hoạt động và một số thời điểm thiếu quan sát. Vì vậy, nhóm xây dựng cửa sổ thời gian dựa trên timestamp thực tế thay vì giả định mọi lượt đo đều cách nhau chính xác 30 giây.

3.2 Đơn vị quan sát và cách lấy mẫu

Nhóm đã cấu hình Task Scheduler cho collector hoạt động từ 03:00 đến 01:00 hôm sau, nghỉ từ 01:00 đến 03:00 hằng ngày. Mỗi dòng dữ liệu tương ứng với một chu kỳ thu thập cho target google, gồm kết quả tổng hợp từ bốn gói ping, một phép phân giải DNS và một yêu cầu HTTPS. Nhóm cấu hình thời gian giữa hai lần bắt đầu chu kỳ là 30 giây; nếu một chu kỳ kéo dài hơn thời gian này, chu kỳ tiếp theo bắt đầu sau khi chu kỳ trước hoàn tất. Timestamp được lấy trước các phép ping, DNS và HTTP nối tiếp nên chúng không được đo đồng thời tuyệt đối. Cửa sổ được tính theo timestamp thực tế.

Bảng 1. Thông tin dữ liệu thu thập

Nội dung	Kết quả kiểm tra
Số dòng và cột nguồn	51.070 dòng, 15 cột
Mẫu đầu	19/08/2026 17:34:00 UTC+7
Mẫu cuối	08/09/2026 01:00:00 UTC+7
Target	google
Trùng dòng hoặc timestamp/target	0
Khoảng cách thông thường	30 giây
Khoảng ngắt trên 60 giây	19, tạo 20 phiên
Khoảng ngắt đúng 60 giây	2

Dữ liệu nhóm thu thập trải dài 19 ngày 7 giờ 26 phút, từ mẫu đầu tiên đến mẫu cuối cùng, đi qua 21 ngày lịch. Trong đó, ngày đầu và ngày cuối chỉ được quan sát một phần thời gian. Nhóm ghi nhận 19 khoảng ngắt kéo dài khoảng hai giờ, phù hợp với lịch nghỉ thu thập từ 01:00 đến 03:00. Các phiên đầy đủ có số đo bắt đầu từ 03:00. Vì những khoảng ngắt này thuộc lịch hoạt động đã thiết lập, nhóm em không xem chúng là dấu hiệu mất mạng.

Khi đối chiếu các thời điểm theo chu kỳ 30 giây, nhóm tính cả mẫu đúng 01:00 vào lịch đo vì dữ liệu thực tế có bản ghi tại thời điểm này. Các mốc ngoài lịch được phân loại là thời gian nghỉ thu thập, không phải dữ liệu bị thiếu trong giờ đo.

Với hơn năm mươi nghìn bản ghi, nhóm sử dụng bộ dữ liệu để thực hiện thống kê mô tả và xây dựng mô hình Isolation Forest. Tuy nhiên, dữ liệu chỉ được thu trên một máy với một cấu hình đích; các cửa sổ trượt còn dùng chung nhiều quan sát nên không hoàn toàn độc lập. Do đó, nhóm giới hạn kết luận trong môi trường thực nghiệm của đề tài, chưa khái quát kết quả cho các nhà mạng, vị trí hoặc điều kiện sử dụng khác.

Để tính tỷ lệ cảnh báo theo ngày, nhóm sử dụng các mẫu đủ lịch sử tạo đặc trưng trong tập kiểm tra của `scored_data.csv`. Đối với trung bình theo giờ, nhóm sử dụng các số đo hợp lệ trong `cleaned_data.csv`. Giờ 01 chỉ có quan sát tại mốc 01:00, còn giờ 02 không có dữ liệu do thuộc lịch nghỉ. Nhóm giữ trống những giờ không được quan sát, thay vì điền giá trị 0 làm sai lệch kết quả thống kê.

3.3 Data dictionary

Bảng 2 mô tả toàn bộ schema. Năm trường `RTT`, `jitter`, `loss`, `DNS` và `HTTP` là nội dung phân tích chính. Các trường còn lại dùng để hiểu hoặc kiểm tra phép đo. Chỉ có một target nên nhóm không cần mã hóa biến phân loại `target_id` thành nhiều cột.

Bảng 2. Từ điển dữ liệu

Trường	Kiểu và đơn vị	Cách hiểu trong bài
timestamp	Thời gian có múi giờ	Mốc bắt đầu lượt đo
target_id	Chuỗi	Nhãn cấu hình đích; ở đây là google
rtt_ms	Số thực, ms	RTT trung bình của phản hồi ping
rtt_min_ms	Số thực, ms	RTT thấp nhất trong lượt
rtt_max_ms	Số thực, ms	RTT cao nhất trong lượt

jitter_ms	Số thực, ms	Trung bình chênh lệch tuyệt đối RTT liên tiếp
n_sent	Số nguyên, gói	Số ping dự kiến gửi; ở đây là 4
n_received	Số nguyên, gói	Số phản hồi đọc được
loss_rate	Số thực, 0 đến 1	Tỷ lệ không nhận phản hồi ping
dns_latency_ms	Số thực, ms	Thời gian phép phân giải tên miền riêng
tcp_connect_ms	Số thực, ms	Bao gồm phân giải hệ thống và tạo TCP
tls_handshake_ms	Số thực, ms	Bao gồm chuẩn bị ngữ cảnh và bước TLS
ttfb_ms	Số thực, ms	Từ sau bước TLS đến byte phản hồi đầu
http_latency_ms	Số thực, ms	Từ bắt đầu kết nối tới khi vòng đọc kết thúc
http_status_code	Số nguyên	Mã phản hồi HTTP đọc được

Ví dụ dòng đầu có RTT 42,75 ms, jitter 0,667 ms, nhận 4/4 phản hồi, loss bằng 0, DNS 42,111 ms và HTTP 512,97 ms. Ý nghĩa của dòng là mô tả một lượt quan sát trong file. HTTP khoảng nửa giây không phải RTT khoảng nửa giây, vì thời gian HTTP gồm nhiều công đoạn hơn một phép ping.

3.4 Làm sạch và kiểm tra chất lượng

```
audit["exact_duplicates"] = int(df.duplicated().sum())

df = df.drop_duplicates().sort_values("timestamp").reset_index(drop=True)

if df.duplicated(["timestamp"]).any():

    raise ValueError "Có hai dòng khác dữ liệu nhưng cùng thời điểm; cần kiểm tra nguồn.")
```

Trong bước tiền xử lý, nhóm em chuyển trường timestamp sang kiểu dữ liệu thời gian theo múi giờ UTC+7 và sắp xếp các bản ghi theo thời gian tăng dần. Chương trình vẫn duy trì hai quy tắc: loại bỏ bản ghi trùng hoàn toàn và dừng xử lý nếu còn các dòng cùng thời điểm nhưng khác dữ liệu để nhóm kiểm tra nguồn. Do dữ liệu chỉ gồm target google, nhóm kiểm tra các bản ghi trùng dựa trên timestamp. Kết quả không phát hiện dòng trùng hoàn toàn hoặc các dòng cùng thời điểm nhưng khác số đo.

Bảng 3. Giá trị thiếu của năm chỉ số chính

Chỉ số	Số giá trị thiếu	Cách xử lý
RTT	2	Giữ thiếu, không gán 0 ms
Jitter	2	Không suy ra độ dao động khi mất hết phản hồi
Loss	0	Giữ giá trị nguồn; đánh dấu một dòng không khớp số đếm
DNS latency	4	Giữ thiếu, đối chiếu nhật ký
HTTP latency	3	Giữ thiếu, không coi là phản hồi nhanh

Tổng cộng có năm bản ghi thiếu ít nhất một chỉ số chính. Trong đó, hai bản ghi lúc 07:52:30 và 07:53:00 ngày 31/08 không nhận được phản hồi ping, nên

RTT và jitter để trống, còn loss_rate bằng 1, tương ứng 100%. Nhóm giữ nguyên các giá trị thiếu này để phản ánh đúng việc không có số đo hợp lệ. File missing_measurements.csv tập hợp riêng những bản ghi cần xem xét. Trong phiên bản dữ liệu hiện tại, không có giá trị jitter nào cần chuyển sang thiếu do số phản hồi nhỏ hơn hai.

Bên cạnh giá trị thiếu, nhóm em kiểm tra tính nhất quán giữa các trường dữ liệu. Hai bản ghi trên có mã trạng thái HTTP 200 nhưng thiếu toàn bộ thời gian TCP, TLS, TTFB và HTTP tổng. Ngoài ra, bản ghi lúc 06:40:00 ngày 27/08 ghi nhận hai phản hồi trong bốn gói gửi, nhưng loss_rate bằng 0. Nếu tính từ số đếm, tỷ lệ tương ứng phải là 0,5. Chương trình đánh dấu ba bản ghi này trong data_quality_notes.csv để đối chiếu thêm, không tự sửa số đếm hoặc tỷ lệ khi chưa xác định được nguyên nhân sai khác.

```
for col in columns:
    data[col] = pd.to_numeric(data[col],
errors="coerce").replace([np.inf, -np.inf], np.nan)
    data.loc[data[col] < 0, col] = np.nan
```

Đối với các chỉ số đo, chương trình chuyển giá trị âm hoặc vô hạn thành giá trị thiếu nếu phát hiện. Riêng các giá trị cao được giữ lại vì chúng có thể phản ánh dấu hiệu suy giảm cần nghiên cứu. Tuy nhiên, giá trị cao cũng có thể xuất hiện do vấn đề trong phép đo; vì vậy, nhóm kết hợp xem xét các chỉ số liên quan và nhật ký trước khi diễn giải nguyên nhân.

Khi tính thống kê trong cửa sổ trượt, nhóm bỏ qua các ô thiếu và chỉ tính mỗi thống kê khi có ít nhất ba giá trị hợp lệ của chỉ số tương ứng. Nếu đặc trưng vẫn còn thiếu, chương trình sử dụng trung vị được tính từ tập huấn luyện để điền vào.

Nhóm chọn trung vị vì dữ liệu chất lượng mạng có thể xuất hiện những giá trị tăng cao đột ngột. So với trung bình, trung vị ít bị ảnh hưởng bởi các giá trị cực đoan nên phù hợp hơn để thay thế ô thiếu bằng một giá trị đại diện. Ví dụ, với các

giá trị RTT là 20, 21, 22, 23 và 500 ms, trung bình bằng 117,2 ms, trong khi trung vị là 22 ms, gần với phần lớn số đo. Điền bằng 0 có thể khiến chương trình hiểu nhầm rằng độ trễ rất thấp; còn giá trị xuất hiện nhiều nhất ít phù hợp với các chỉ số thời gian liên tục vì nhiều số đo có thể chỉ xuất hiện một lần.

Trong bài, trung vị chỉ dùng để điền đặc trưng thiếu, không phải ngưỡng xác định bất thường; việc phát hiện mẫu khác thường do Isolation Forest thực hiện. Cách điền này đơn giản nhưng có thể làm giảm biểu hiện bất thường nếu dữ liệu thiếu xảy ra đúng lúc mạng suy giảm. Vì vậy, nhóm vẫn giữ thông tin về các lượt đo thiếu để xem xét riêng.

Trung vị chỉ được tính từ tập huấn luyện, không sử dụng tập kiểm tra hoặc dữ liệu tương lai để điền ngược. Trong lần thực nghiệm hiện tại, sau khi tạo cửa sổ và loại các mẫu đầu phiên chưa đủ lịch sử, không còn ô đặc trưng nào cần điền thay thế. Do đó, lựa chọn trung vị chưa tác động đến kết quả của lần chạy này.

Cuối cùng, nhóm em chia dữ liệu thành các phiên đo (session), với một phiên mới bắt đầu khi khoảng cách giữa hai lượt liên tiếp lớn hơn 60 giây. File `cleaned_data.csv` vẫn giữ 51.070 bản ghi sau bước làm sạch. Việc loại các mẫu đầu phiên chưa đủ lịch sử chỉ được thực hiện ở bước tạo đặc trưng, giúp phân biệt rõ dữ liệu đã kiểm tra với dữ liệu đủ điều kiện đưa vào mô hình.

3.5 Tạo đặc trưng theo thời gian

Để tránh đánh giá chỉ từ một lần ping, nhóm dùng cửa sổ trượt 5 phút. Tại thời điểm t , cửa sổ là $(t - 5 \text{ phút}, t]$, tức chỉ dùng hiện tại và quá khứ. Với khoảng lấy mẫu 30 giây, khi đã đủ lịch sử mỗi cửa sổ có 10 dòng. Nhóm bỏ hai dòng đầu mỗi phiên khỏi phân mô hình vì chưa đủ ba thời điểm. Với 20 phiên, loại 40 dòng khởi động và còn 51.030 dòng đặc trưng.

Bảng 4. Chín đặc trưng đầu vào mô hình

Đặc trưng	Số cột	Mục đích
Trung bình RTT, jitter, loss, DNS, HTTP trong 5 phút	5	Mô tả mức chất lượng gần thời điểm xét
Độ lệch chuẩn RTT trong 5 phút	1	Mô tả mức dao động giữa các lượt
P95 RTT trong 5 phút	1	Phản ánh phần RTT cao trong cửa sổ
P95 HTTP trong 5 phút	1	Phản ánh phần HTTP latency cao
Giờ cộng phút chia 60	1	Bổ sung bối cảnh giờ trong ngày

Độ lệch chuẩn RTT giữa các lượt khác với jitter trong một lượt. Chẳng hạn, một lượt có các phản hồi gần nhau nên jitter thấp, nhưng trung bình RTT của nhiều lượt vẫn có thể tăng dần. Việc giữ hai đặc trưng này giúp quan sát hai mức biến động khác nhau.

Nhóm huấn luyện Isolation Forest bằng các đặc trưng được tạo từ số đo và giờ trong ngày, không gán trước nhãn bình thường hoặc bất thường theo các khung giờ.

Nhóm em không dùng StandardScaler cho Isolation Forest vì mô hình chia dữ liệu theo từng đặc trưng bằng cây, không dựa vào khoảng cách Euclid như K-Means. Việc so sánh mức tăng giữa các chỉ số ở Chương 5 có phép chia cho độ phân tán riêng; đó là bước diễn giải, không phải bước chuẩn hóa đầu vào mô hình.

3.6 Thiết kế đánh giá Isolation Forest

Sau khi tạo feature theo chiều thời gian, nhóm dùng 70% dòng đầu cho train và 30% cuối cho test. Train có 35.721 dòng, kết thúc lúc 07:23:00 ngày 02/09/2026. Test có 15.309 dòng, bắt đầu lúc 07:23:30 cùng ngày. Bộ điền thiếu và Isolation Forest chỉ fit trên train; test được giữ để xem mô hình phản ứng với giai đoạn sau.

Cửa sổ đầu test có thể sử dụng lịch sử ở cuối train vì các mẫu đó đã có tại thời điểm quyết định. Nó không lấy số liệu tương lai. Các cửa sổ vẫn chồng lấp nên nhóm không tính khoảng tin cậy theo giả định mọi dòng độc lập, và không trộn ngẫu nhiên train/test.

Do chưa có thông tin xác nhận riêng về thời điểm mạng thực sự gặp sự cố, nhóm đánh giá kết quả thông qua điểm bất thường (anomaly score), tỷ lệ cảnh báo, số khoảng cảnh báo liên tiếp, thời lượng ước lượng của các khoảng và mức tăng của từng chỉ số mạng. Nhóm xem xét cụ thể một khoảng cảnh báo dài, một cảnh báo ngắn và một mẫu không bị đánh dấu để kiểm tra mức độ phù hợp của kết quả với các số đo mạng, chưa coi đây là căn cứ xác nhận cảnh báo đúng hoặc sai.

Nhóm chỉ dùng baseline làm mốc đối chiếu; số mẫu trùng nhau thể hiện mức đồng thuận, không phải số dự đoán đúng.

CHƯƠNG 4: XÂY DỰNG CHƯƠNG TRÌNH VÀ THỰC NGHIỆM

4.1 Kiến trúc chương trình

Nhóm sử dụng hai file Python: `collector.py` thu số đo, `phan_tich.py` đọc CSV và phân tích. Mỗi bước nằm trong một hàm để nhóm có thể lần theo từ dữ liệu đến kết quả.

Nhóm thực nghiệm trên CSV có sẵn: làm sạch, tạo đặc trưng, chạy Isolation Forest, đối chiếu baseline, gộp khoảng và vẽ biểu đồ. Phân tích không phát sinh phép đo mạng.

Bảng 5. Các thành phần của bộ chương trình

Thành phần	Chức năng
collector.py	Thu định kỳ theo chu kỳ 30 giây; chạy liên tục và dừng bằng Ctrl+C
phan_tich.py	Toàn bộ pipeline phân tích và mô hình
du_lieu	Chứa CSV 51.070 dòng và log đối chiếu
ket_qua	Chứa bảng số liệu, điểm bất thường, khoảng cảnh báo và hình
README.md	Cài môi trường, lệnh chạy, kết quả mong đợi

Trong `phan_tich.py`, `read_and_clean` kiểm tra dữ liệu; `make_features` tạo cửa sổ; `build_intervals` gộp cảnh báo; `make_figures` tạo hình. Phân tích HTTP bổ sung nằm trong một hàm riêng. Nhóm trích đoạn mã cần giải thích trong báo cáo; source đầy đủ nộp kèm.

4.2 Thu thập định kỳ và ghi nhật ký

`collector.py` gửi bốn ping, thực hiện một phép DNS rồi gửi một yêu cầu HTTPS. Chương trình chỉ chạy liên tục theo `INTERVAL_SECONDS` và dừng

bằng Ctrl+C. Nhóm giữ nguyên ba hàm đo; dọn phần mô tả và import thừa cho khớp hành vi.

```
while True:
    cycle_start = time.time()

    rows = collect_once_all_targets()
    for row in rows:
        append_csv(row)
        logging.info(f"Đã ghi: {row}")

    elapsed = time.time() - cycle_start
    sleep_time = max(0, INTERVAL_SECONDS - elapsed)
    time.sleep(sleep_time)
```

Task Scheduler quản lý giờ hoạt động 03:00 đến 01:00 hôm sau; collector quản lý chu kỳ trong thời gian đó. Nhóm không chạy chồng nhiều collector. `phan_tich.py` chỉ đọc CSV ngoại tuyến, không tạo traffic mạng.

Chu kỳ cấu hình là 30 giây, dự kiến khoảng tám ping, hai phép DNS và hai yêu cầu HTTPS mỗi phút với một target. Đây là số thao tác logic, không phải tổng số gói. Nếu lượt đo quá 30 giây, thời gian nghỉ bằng 0 và lượt tiếp theo bắt đầu sau khi lượt trước kết thúc. Không chạy bù những lượt bị lỡ.

Log đối chiếu có 51.070 dòng số đo và 20 thông báo khởi động ở mức INFO. Nhóm chưa kết luận mạng luôn tốt chỉ vì không có WARNING hay ERROR. Nhóm chỉ thực hiện các phép thu trên máy và mạng được phép; không quét cổng, thu dữ liệu cá nhân hoặc tạo tải để gây suy giảm.

4.3 Thực hiện bước làm sạch

`read_and_clean` kiểm tra có đủ timestamp và năm chỉ số tối thiểu. Sau khi đọc thời gian, chương trình sắp xếp theo timestamp và phát hiện các dòng trùng. Nhóm thực hiện các kiểm tra này để bảo đảm thứ tự thực sự đúng trong cửa sổ, không chỉ để file trông gọn hơn.

```
bad = (df.n_received < 2) & df.jitter_ms.notna()
df.loc[bad, "jitter_ms"] = np.nan
```

Đoạn mã trích lược minh họa quy tắc quan trọng. Khi số phản hồi nhỏ hơn 2, không đủ thông tin tính chênh lệch giữa hai RTT. Nhóm dùng NaN để giữ đúng nghĩa chưa đo được. Nếu thay bằng 0, chương trình sẽ hiểu nhầm lượt ít phản hồi là lượt có độ dao động rất thấp.

Sau làm sạch, `cleaned_data.csv` giữ 51.070 dòng. File mới không có jitter cần sửa vì số phản hồi dưới hai; những ô RTT, jitter, DNS và HTTP đã thiếu được giữ đúng nghĩa. Chương trình xuất riêng năm lượt có chỉ số thiếu và ba lượt có điểm không nhất quán để đối chiếu. Nhóm không xóa dòng chỉ vì RTT hoặc HTTP cao.

`summary.json` lưu số dòng, giá trị thiếu, khoảng thời gian, cấu hình và phiên bản thư viện. Nhóm em nộp kèm CSV và log để đối chiếu.

4.4 Tính cửa sổ và đặc trưng

```
roll = group[METRICS].rolling("5min", min_periods=3)
features = roll.mean().add_suffix("_mean")
features["rtt_std"] = roll["rtt_ms"].std(ddof=0)
features["http_p95"] = roll["http_latency_ms"].quantile(0.95)
```

Ở đây, group chứa dữ liệu của một phiên đo, trong đó nhóm đặt timestamp làm chỉ mục để tính cửa sổ theo thời gian. Với `rolling("5min")`, tại mỗi thời điểm, chương trình lấy các bản ghi trong năm phút gần nhất, gồm bản ghi hiện tại và các bản ghi trước đó còn nằm trong cửa sổ. Vì vậy, cửa sổ được xác định theo timestamp thực tế, không phải một số dòng cố định. Khi đo đều mỗi 30 giây, một cửa sổ đầy đủ thường có 10 bản ghi; nếu thiếu lượt đo hoặc mới bắt đầu phiên, số bản ghi có thể ít hơn.

Tham số `min_periods=3` yêu cầu mỗi phép thống kê phải có ít nhất ba giá trị hợp lệ của chỉ số đang xét. Chẳng hạn, nếu cửa sổ có bốn bản ghi nhưng chỉ có hai giá trị RTT hợp lệ, chương trình chưa tính trung bình RTT mà trả về giá trị

thiếu (NaN). Điều kiện này được kiểm tra riêng cho từng chỉ số, nên cùng một cửa sổ có thể tính được trung bình HTTP nhưng chưa tính được trung bình RTT.

Ngoài các giá trị trung bình và độ lệch chuẩn, nhóm còn tính P95 RTT để mô tả mức RTT cao trong cửa sổ. Đặc trưng giờ được tính bằng giờ cộng với phút chia cho 60; ví dụ, 08:30 tương ứng với 8,5. Đặc trưng này bổ sung thông tin về thời điểm đo trong ngày. Các đặc trưng được tổng hợp trong Bảng 4.

Ví dụ khi xét lúc 20:03:00 với lịch đều 30 giây, cửa sổ dùng từ 19:58:30 đến 20:03:00. Những giá trị trước 20:00 có thể vẫn còn bình thường. Do đó, ngay sau khi bắt đầu đoạn suy giảm, trung bình 5 phút có thể chưa tăng đủ cao để vượt ngưỡng. Ngược lại, sau lúc kết thúc, cửa sổ vẫn giữ một phần giá trị trước đó và cảnh báo có thể kéo dài thêm.

Ảnh hưởng này là đánh đổi của phép làm mượt. Cửa sổ ngắn phản ứng nhanh nhưng nhạy với dao động đơn lẻ. Cửa sổ dài giảm dao động nhưng phản ứng chậm hơn. Nhóm chọn 5 phút làm cấu hình chính để giữ cách tính dễ hiểu, không khẳng định đây là cửa sổ tối ưu cho mọi mục đích sử dụng.

4.5 Huấn luyện Isolation Forest

```
imputer.fit(train_features)
model = IsolationForest(
    n_estimators=100, max_samples=256,
    contamination=0.05, random_state=42, n_jobs=1)
model.fit(imputer.transform(train_features))
```

Đoạn mã trích lược thể hiện việc chỉ học từ train. Bộ điền thiếu tính trung vị trên train; mô hình nhận các đặc trưng đã xử lý của train. Khi chuyển sang test, nhóm chỉ gọi transform và dự đoán, không gọi fit lại. Cách thực hiện này tránh đưa thông tin của tập test vào bước tiền xử lý và huấn luyện. [10] Nhóm chọn chia theo thời gian để mô phỏng việc sử dụng dữ liệu ở giai đoạn trước nhằm phân tích giai đoạn sau.

Mô hình có 100 cây, mỗi cây dùng tối đa 256 mẫu để xây dựng các phép chia. `random_state=42` giúp giữ tính lặp lại trong cùng môi trường. Nhóm chọn `contamination=0,05` cho cấu hình chính. Sau khi huấn luyện, ngưỡng tương ứng trên thang điểm của bài là khoảng 0,601695. Mẫu có điểm cao hơn ngưỡng này bị đánh dấu bất thường.

Nhóm lưu điểm bằng âm của `score_samples`, để giá trị lớn hơn dễ hiểu là khác thường hơn. Điểm này không phải xác suất mất mạng. Một mẫu có điểm 0,7 không có nghĩa xác suất sự cố bằng 70%. Nhóm chuyển nhãn của mô hình từ `-1` thành `True` để thuận tiện thống kê và xuất CSV.

Nhóm thực hiện thí nghiệm bổ sung để xem tỷ lệ cảnh báo thay đổi như thế nào với các ngưỡng tương ứng `contamination` 0,02; 0,05 và 0,10. Các ngưỡng được xác định từ phân vị điểm bất thường trên tập huấn luyện của cùng một mô hình Isolation Forest đã huấn luyện. Nhóm chỉ dùng kết quả này để phân tích ảnh hưởng của ngưỡng, không lựa chọn cấu hình nhằm tăng số cảnh báo trên tập kiểm tra.

Bảng 6. Số lượng và tỷ lệ cảnh báo trên tập kiểm tra theo các ngưỡng tương ứng với contamination

contaminatio n	threshold_from_trai n	test_flag s	test_rate
0.02	0.6646818360361673	16	0.001045136847605983 3
0.05	0.6016949948667434	402	0.026259063296100334
0.1	0.5411673839698602	1329	0.086811679404272

4.6 Ngưỡng đơn giản dùng để đối chiếu

Baseline dùng năm giá trị trung bình trượt. Một mẫu được đánh dấu nếu ít nhất một chỉ số vượt ngưỡng ở Bảng 7. Dấu so sánh là lớn hơn nghiêm ngặt. Chẳng hạn loss trung bình đúng 0,05 chưa vượt ngưỡng 5%, còn 0,075 thì vượt.

Bảng 7. Ngưỡng baseline trong thực nghiệm

Chỉ số trung bình 5 phút	Ngưỡng cảnh báo
RTT	Trên 100 ms
Jitter	Trên 20 ms
Tỷ lệ không nhận phản hồi ICMP	Trên 5%
DNS latency	Trên 100 ms
HTTP latency	Trên 1.000 ms

Nhóm chọn các ngưỡng làm quy tắc minh họa cho bài, không lấy từ một cam kết chất lượng dịch vụ cụ thể. Ưu điểm của baseline là giải thích được trực tiếp: có chỉ số trung bình nào đã vượt mức. Nhược điểm là một mẫu khác thường nhưng chưa vượt ngưỡng sẽ không được đánh dấu, còn mức bình thường của một đường truyền xa có thể đã vượt ngưỡng RTT.

Nhóm đánh giá baseline và Isolation Forest trên những timestamp sau giai đoạn khởi động và cùng tập test. Nhóm không dùng kết quả baseline để huấn luyện mô hình hoặc làm nhãn đáp án. Điều này tránh việc đánh giá mô hình chỉ bằng mức độ bất thường quy tắc.

4.7 Gộp thành khoảng thời gian và giải thích chỉ số

Mỗi mẫu được đánh dấu vẫn chỉ là một thời điểm. Để tạo kết quả phù hợp mục tiêu T19, nhóm dùng chương trình nối các mẫu cảnh báo liên tiếp cách nhau đúng 30 giây. Chuỗi phải có ít nhất ba mẫu mới được xuất thành một khoảng. Khi gặp mẫu bình thường, thiếu một timestamp hoặc chuyển train/test, chuỗi kết thúc.

Thời điểm bắt đầu là timestamp đầu chuỗi. Mốc kết thúc ước lượng là timestamp cuối cộng 30 giây, được đặt tên rõ là `end_exclusive_estimated`. Ví dụ ba mẫu ở 20:00:00, 20:00:30 và 20:01:00 tạo độ phủ ước lượng đến trước

20:01:30. Thời lượng 1,5 phút này mô tả vùng hỗ trợ của mẫu, không khẳng định sự cố thật kéo dài chính xác 90 giây.

```
ref_median = train[means].median()
ref_iqr = train[means].quantile(.75)-train[means].quantile(.25)
# Loss thường có IQR=0; dùng std train thay thế, tránh chia cho 0.
ref_iqr = ref_iqr.where(ref_iqr > 0,
train[means].std(ddof=0)).replace(0, 1)
```

Để tìm chỉ số tăng mạnh nhất, nhóm dùng chương trình tính trung bình các đặc trưng mean trong từng khoảng, trừ trung vị train của chỉ số tương ứng rồi chia cho IQR train. IQR là khoảng cách giữa P75 và P25. Nếu IQR bằng 0, dùng độ lệch chuẩn train; nếu std vẫn bằng 0, dùng một đơn vị để tránh phép chia không xác định. Chỉ số có mức tăng chuẩn hóa lớn nhất được ghi trong intervals.csv.

Phép so sánh này giúp tránh chọn HTTP chỉ vì đơn vị mili giây của nó thường lớn hơn RTT. Tuy vậy, đây là mô tả độ tăng so với mức tham chiếu, không phải độ quan trọng nội bộ của feature trong Isolation Forest và không phải bằng chứng nguyên nhân. Nếu không có chỉ số nào tăng, khoảng được ghi là chưa có bằng chứng tăng độ trễ/mất phản hồi theo cách so sánh đó.

4.8 Phân tích bổ sung các thành phần HTTP

Sau khi Isolation Forest đã xuất điểm và khoảng cảnh báo, hàm `analyze_http_components` đọc riêng TCP, TLS, TTFB và HTTP tổng. Ba thành phần bổ sung không nằm trong METRICS và không được thêm vào chín feature của mô hình.

Nhóm chỉ sử dụng lượt có đủ bốn thời gian hợp lệ, không điền ô thiếu để tính tương quan. Có 51.067 lượt phù hợp, loại ba lượt thiếu. Pearson mô tả quan hệ tuyến tính; Spearman mô tả sự cùng tăng hoặc giảm theo thứ hạng. Nhóm tính hai hệ số trên cùng các dòng để so sánh được.

Theo các mốc thời gian của collector, HTTP còn có bước đọc sau byte đầu. Chương trình tính phần này bằng HTTP tổng trừ TCP, TLS và TTFB. Đây là thời gian còn lại đến khi vòng đọc kết thúc, chưa chắc toàn bộ phản hồi đã tải xong do giới hạn đọc và timeout. Không có lượt đủ dữ liệu nào cho phần dư âm vượt sai số làm tròn 0,01 ms.

```
valid["remaining_read_ms"] = (  
    valid["http_latency_ms"] - valid[parts].sum(axis=1)  
)  
pearson = valid[columns].corr(method="pearson")  
spearman = valid[columns].corr(method="spearman")
```

Để giải thích một khoảng, nhóm dùng chương trình chọn khoảng IF dài nhất trên test, đối chiếu trung bình số đo gốc của từng thành phần với các lượt thuộc train đủ dữ liệu. Train là mức tham chiếu, không mặc định mọi lượt đều bình thường. Nhóm xem thêm lượt HTTP cao nhất toàn CSV để phân biệt tăng vọt đơn lẻ với đợt chậm kéo dài. Đây là phân tích mô tả, không phải đánh giá độ chính xác mới.

4.9 Cách tái lập và kiểm tra

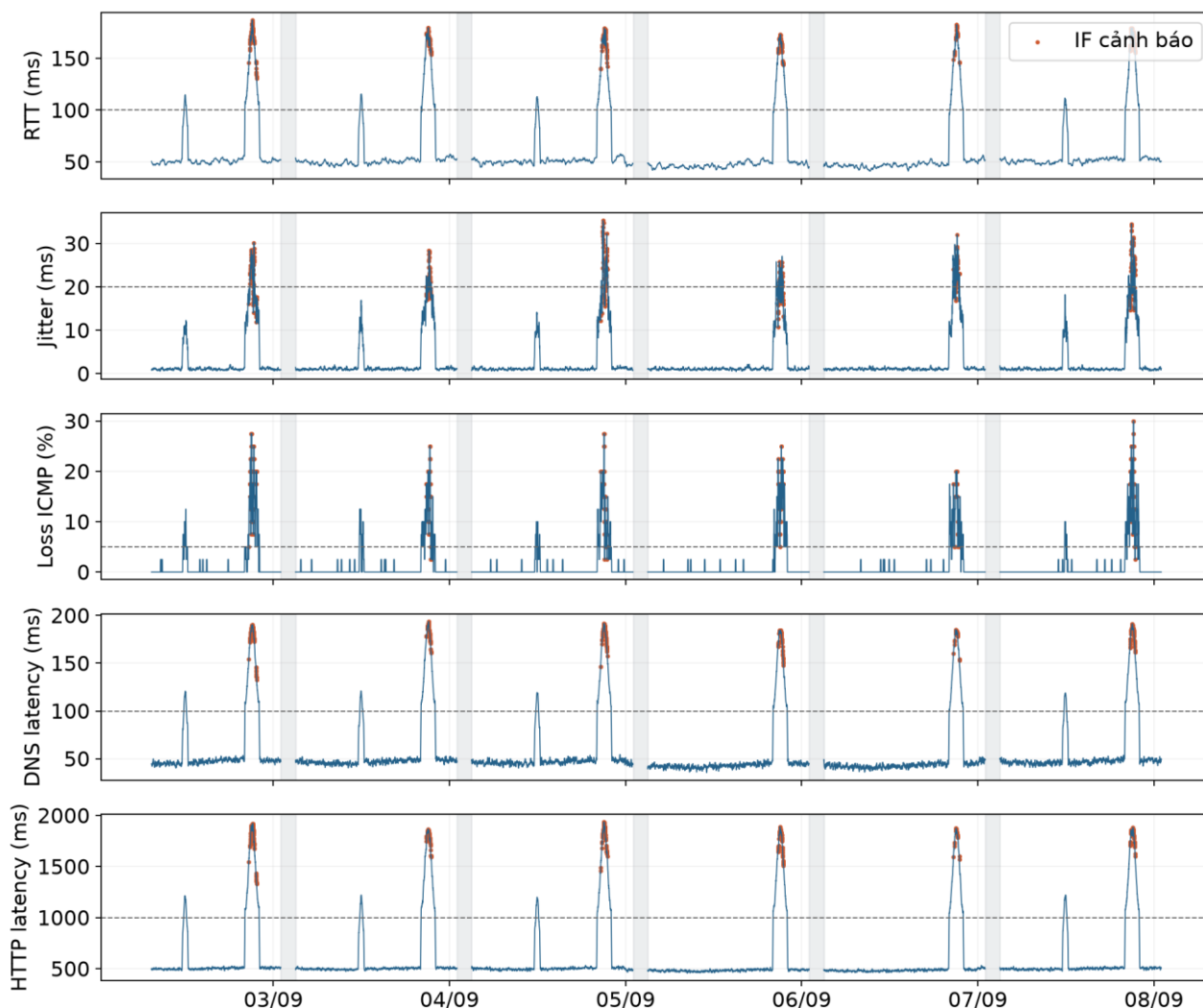
Nhóm thực nghiệm với Python 3.12.14, pandas 3.0.1, numpy 2.5.3, scikit-learn 1.9.0 và matplotlib 3.11.1. Cài requirements.txt rồi chạy python phan_tich.py. Target Google, cửa sổ 5 phút và contamination 0,05 được cố định trong mã. Chương trình đọc du_lieu/network_quality_data.csv, xuất vào ket_qua; chỉ hai tùy chọn --input và --output dùng để đổi vị trí file.

Toàn bộ mã nguồn, hướng dẫn chạy và các tệp kết quả của đề tài được lưu tại GitHub: https://github.com/biay3x/Mang_may_tinh

CHƯƠNG 5: KẾT QUẢ VÀ THẢO LUẬN

5.1 Timeline cảnh báo Isolation Forest

Hình 1. Cảnh báo Isolation Forest trên tập kiểm tra
Cửa sổ 5 phút; vùng xám chưa đủ dữ liệu; nét đứt ngưỡng đối chiếu



Hình 1. Cảnh báo IF trên năm chỉ số trung bình trượt ở tập kiểm tra; vùng xám chưa có dữ liệu cửa sổ.

Nhóm quan sát thấy các cụm cảnh báo nằm ở phần tăng cao của nhiều chỉ số. Vùng xám gồm lịch nghỉ thu 01:00-03:00 và đầu phiên chưa đủ mẫu; không được đọc thành khoảng mạng suy giảm. Các đường ngắt giữa phiên giúp tránh suy diễn về thời gian không đo.

5.2 Điểm bất thường và các khoảng được phát hiện

Nhóm ghi nhận IF đánh dấu 402/15.309 mẫu test, tương đương 2,63%, với ngưỡng điểm 0,601695 học từ train. Điểm lớn hơn thể hiện khác thường hơn, không phải xác suất mất mạng. Phân bố điểm được trình bày ở Hình 4 trong phụ lục.

Nhóm xác định 21 khoảng gồm ít nhất ba cảnh báo liên tiếp cách nhau 30 giây. Tổng 371 mẫu thuộc các khoảng có độ phủ ước lượng 185,5 phút; 31 cảnh báo còn lại thuộc chuỗi ngắn hơn ba mẫu. Đây là độ phủ từ số đo, chưa phải thời lượng sự cố đã xác nhận.

Bảng 8. Bốn khoảng IF dài nhất trên test

Ngày	Bắt đầu	Đến trước	Phút ước lượng
02/09	20:54:30	21:23:00	28,5
04/09	20:51:00	21:19:30	28,5
03/09	20:57:30	21:25:00	27,5
07/09	20:52:30	21:18:30	26,0

Khoảng 02/09 từ 20:54:30 đến trước 21:23:00 có 57 mẫu. Trung bình trượt RTT là 174,09 ms, DNS 184,77 ms, HTTP 1.857,10 ms và loss 16,80%. Các chỉ số cùng tăng là cơ sở để nhóm chọn khoảng này để xem xét chi tiết ở mục 5.5.

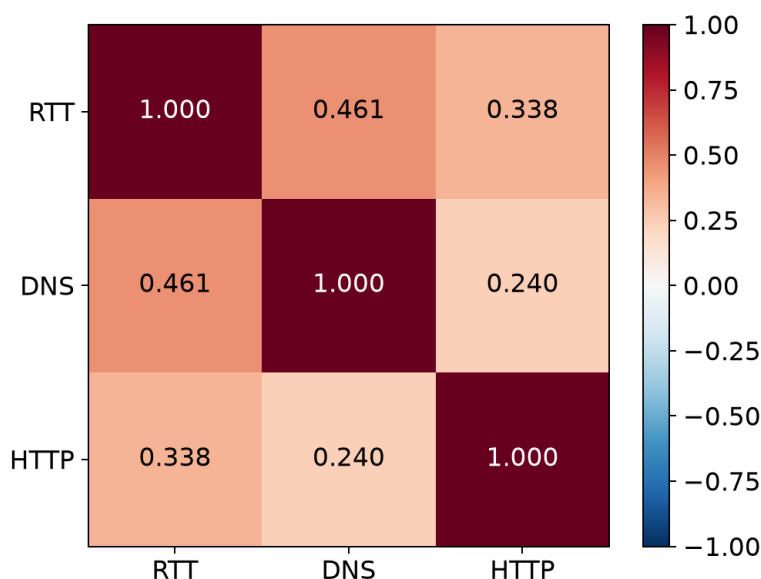
5.3 Giải thích cảnh báo và trường hợp gần ngưỡng

Nhóm IF cảnh báo có RTT trung bình trượt 167,17 ms, HTTP 1.786,51 ms; nhóm không cảnh báo tương ứng 55,91 ms và 565,68 ms. Theo mức tăng chia cho độ phân tán train, HTTP tăng trội ở 20/21 khoảng và jitter ở một khoảng. Đây là mô tả biến động, không phải độ quan trọng nội bộ của feature hoặc xác nhận nguyên nhân.

Ngày 07/09 lúc 20:52:00 có điểm 0,601672, thấp hơn rất ít so với ngưỡng 0,601695, nên IF không cảnh báo dù HTTP trung bình trượt là 1.832,07 ms. Từ mẫu này, nhóm nhận thấy không bị đánh dấu không có nghĩa mạng tốt. Nhấn gần ngưỡng nhảy với dữ liệu và tham số; cần đọc cả số đo và chuỗi thời gian.

5.4 Quan hệ RTT DNS và HTTP latency

Hình 2. Tương quan Pearson giữa RTT, DNS và HTTP trên các cặp giá trị hợp lệ.



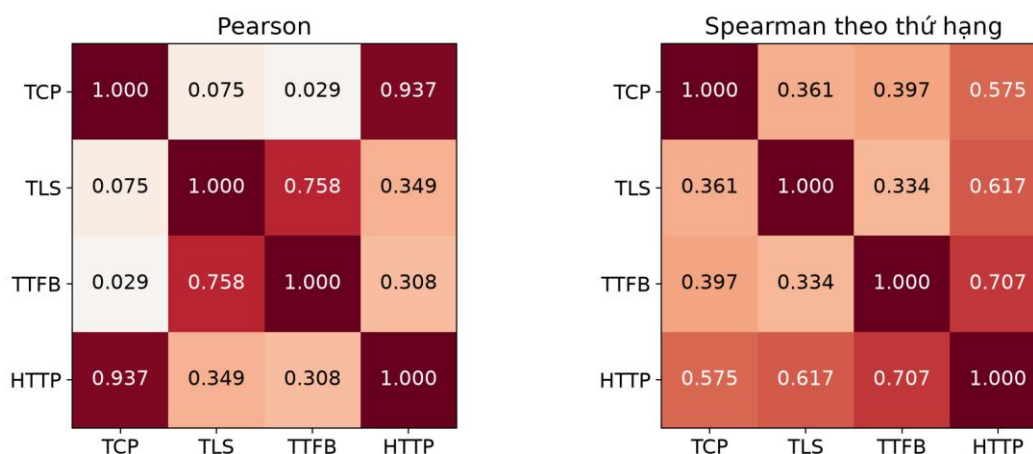
Hình 2. Tương quan Pearson giữa RTT, DNS và HTTP trên các cặp giá trị hợp lệ.

Pearson RTT-DNS khoảng 0,4605, RTT-HTTP 0,3378 và DNS-HTTP 0,2396. Nhóm nhận thấy các chỉ số có đồng biến nhưng mức liên hệ chưa chặt. Ping tới 8.8.8.8, còn HTTP tới địa chỉ phân giải từ www.google.com. Phép DNS được đo riêng, nên nhóm chưa thể kết luận DNS gây HTTP tăng chỉ từ tương quan, hoặc tính HTTP bằng RTT cộng cột DNS.

5.5 Phân tích nâng cao khi HTTP tăng cao

Nhóm xem TCP, TLS và TTFB để giải thích HTTP, không đưa thêm chúng vào Isolation Forest. Dữ liệu gồm 51.067 lượt đủ cả bốn thời gian; ba lượt thiếu bị bỏ riêng trong phép phân tích này. Hình 3 thể hiện tương quan giữa ba thành phần và quan hệ của từng thành phần với HTTP tổng.

Hình 3. Pearson và Spearman của TCP, TLS, TTFB và HTTP trên cùng các lượt đủ dữ liệu



Hình 3. Pearson và Spearman của TCP, TLS, TTFB và HTTP trên cùng các lượt đủ dữ liệu

Giữa ba thành phần, Pearson TCP-TLS là 0,0754, TCP-TTFB 0,0286 và TLS-TTFB 0,7580. Nhóm nhận thấy TCP không thường biến động tuyến tính cùng hai bước còn lại, trong khi TLS và TTFB có liên hệ rõ hơn trên toàn file. Spearman tương ứng 0,3612; 0,3970; 0,3341, cho thấy kết luận còn phụ thuộc cách đo quan hệ và các giá trị cực cao.

Với HTTP tổng, Pearson TCP-HTTP đạt 0,9369, nhưng Spearman chỉ 0,5752. Ngược lại, Spearman TTFB-HTTP đạt 0,7068, cao hơn TCP-HTTP. Vì vậy, nhóm không lấy hệ số Pearson lớn nhất để khẳng định mọi lần HTTP chậm đều do TCP. HTTP tổng vốn chứa các thành phần này, nên tương quan còn chịu ảnh hưởng của quan hệ thành phần-tổng.

Bảng 9. Đối chiếu số đo gốc của các thành phần trong khoảng IF ngày 02/09

Thành phần	Train ms	Khoảng IF ms	Mức tăng ms
TCP	103,91	257,57	153,66
TLS	117,38	371,79	254,42
Chờ byte đầu	203,24	666,36	463,12
Đọc phần còn lại	192,24	551,76	359,52
HTTP tổng	616,77	1.847,48	1.230,71

Nhóm lấy mức tham chiếu là trung bình 35.718 lượt thuộc train đủ dữ liệu; khoảng IF có 57 lượt. HTTP tăng từ 616,77 lên 1.847,48 ms. TTFB tăng nhiều nhất, khoảng 463,12 ms, tiếp theo là đọc phần còn lại 359,52 ms, TLS 254,42 ms và TCP 153,66 ms. Khoảng chậm này biểu hiện tăng ở nhiều bước; thời gian chờ byte đầu đóng góp lớn nhất vào mức tăng theo cách đối chiếu trên. Bảng dùng số đo gốc nên khác trung bình trượt tại mục 5.2.

Nhóm xem thêm trường hợp 31/08 lúc 09:12:00: HTTP đạt 47.294,06 ms, riêng TCP 46.899,46 ms, chiếm 99,17%. Ở lượt này, thời gian chủ yếu nằm trong bước tạo kết nối. Đây là mẫu thuộc giai đoạn train, nhóm dùng để giải thích cực trị, không dùng làm kết quả đánh giá test. Hình 7 trong phụ lục cho thấy hai trường hợp có cơ cấu thời gian rất khác nhau.

Trong collector, TCP còn gồm phân giải tên miền của hệ thống; TLS gồm chuẩn bị ngữ cảnh và bắt tay; TTFB gồm gửi yêu cầu và chờ byte đầu. TTFB cao có thể liên quan đến mạng hoặc xử lý phía máy chủ, chưa xác định được bên nào gây chậm. Phần đọc còn lại kết thúc khi hết phản hồi, timeout hoặc đạt giới hạn

đọc; không phải phép đo riêng tốc độ tải. Nhóm chỉ kết luận về bước ghi nhận thời gian tăng, chưa xác định nguyên nhân gốc ở thiết bị hoặc dịch vụ nào.

5.6 Baseline đối chiếu và giới hạn

Baseline đánh dấu 1.624 mẫu test; cả 402 mẫu IF đều thuộc tập này. Có 1.222 mẫu chỉ baseline đánh dấu và 13.685 mẫu cả hai bỏ qua. Jaccard $402/1.624 = 0,2475$ mô tả mức giao nhau, không phải độ chính xác. Nhóm chỉ dùng baseline làm mốc đối chiếu.

CSV và log có số đo khớp nhau nhưng chưa có nhãn sự cố độc lập, nên nhóm không tính F1. Lịch nghỉ ban đêm giới hạn thời gian quan sát; hai mốc trong giờ chạy thiếu mẫu và ba dòng không nhất quán cần xem lại. Cửa sổ chồng lấp, chỉ một target và các phép đo khác đích hạn chế khả năng suy rộng. Kết quả phù hợp để chọn khoảng cần kiểm tra và bước HTTP cần xem sâu hơn, chưa đủ tự xác nhận vi phạm chất lượng dịch vụ.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Nhóm em đã xây dựng được chương trình phân tích dữ liệu mạng theo thời gian bằng Isolation Forest. Phiên bản hiện tại dùng 51.070 dòng, tách 20 phiên và tạo chín feature từ năm chỉ số chính. Bộ mã nguồn được tổ chức thành `collector.py` và `phan_tich.py`; baseline theo ngưỡng chỉ dùng để đối chiếu.

Nhóm ghi nhận mô hình đánh dấu 402 mẫu test, gộp thành 21 khoảng với độ phủ ước lượng 185,5 phút. HTTP tăng trội trong 20 khoảng và jitter trong một khoảng theo phép so sánh chuẩn hóa mô tả. Các khoảng và điểm số giúp chọn thời gian cần xem lại, chưa phải sự cố thật đã xác nhận.

Qua phân tích nâng cao, nhóm nhận thấy cần đọc riêng từng bước khi HTTP tăng. Lướt cục đại ngày 31/08 có phần lớn thời gian trong TCP, trong khi khoảng IF ngày 02/09 có mức tăng TTFB lớn nhất so với tham chiếu train. Tương quan giữa các thành phần hỗ trợ giải thích nhưng không chứng minh nguyên nhân. Đây là phần phân tích sau mô hình; đầu vào Isolation Forest không thay đổi.

Nhóm đã dùng log để đối chiếu các bản ghi, còn việc đánh giá đúng/sai cần nguồn xác nhận sự cố riêng. Kết quả trả lời được thời gian cần xem xét, chỉ số tăng rõ và bước HTTP ghi nhận phần tăng, trong giới hạn của một máy, một cấu hình đích và lịch thu theo ngày.

6.2 Hướng phát triển

Ưu tiên tiếp theo của nhóm là lưu dữ liệu nguyên trạng cùng nhật ký collector và ghi lại sự cố quan sát được. Nhóm cần giữ file thô và lịch sử phiên bản, giải thích phân ghép/trùng nếu có và ghi nhận sự cố quan sát được. Khi đó có thể đánh giá đúng/sai và độ trễ phát hiện của các khoảng.

Sau khi nguồn dữ liệu rõ ràng hơn, nhóm có thể bổ sung đích đo được phép, đánh dấu HTTP đọc chưa hoàn tất và tách validation theo thời gian để lựa chọn cửa sổ/ngưỡng. Việc mở rộng nên thực hiện từng bước, ưu tiên chất lượng phép đo và khả năng giải thích trước khi tăng độ phức tạp mô hình.

TÀI LIỆU THAM KHẢO

Các tài liệu trực tuyến dưới đây đã được đối chiếu ngày 08/09/2026. Các RFC được dùng cho khái niệm gốc liên quan trực tiếp tới đề tài; không suy diễn rằng mọi nội dung lịch sử của RFC đều là hướng dẫn vận hành hiện hành. Đường dẫn tài liệu thư viện có thể cập nhật theo thời gian; phiên bản thư viện thực nghiệm phải ghi riêng theo môi trường thực sự chạy.

[1] J. Postel (1981). *Internet Control Message Protocol*. RFC 792.

<https://www.rfc-editor.org/rfc/rfc792.html>. Dùng cho ICMP Echo Request/Echo Reply và giới hạn của phép đo ICMP.

[2] G. Almes, S. Kalidindi và M. Zekauskas (1999). *A Round-trip Delay Metric for IPPM*. RFC 2681. <https://www.rfc-editor.org/info/rfc2681/>. Dùng cho RTT và sự khác biệt giữa thời gian khứ hồi với độ trễ một chiều.

[3] C. Demichelis và P. Chimento (2002). *IP Packet Delay Variation Metric for IP Performance Metrics (IPPM)*. RFC 3393. <https://www.rfc-editor.org/info/rfc3393/>. Dùng để phân biệt IP delay variation một chiều với độ dao động RTT tự tính trong collector.

[4] P. Mockapetris (1987). *Domain Names — Concepts and Facilities*. RFC 1034. <https://www.rfc-editor.org/info/rfc1034/>. Dùng cho tổ chức DNS, resolver và bộ nhớ đệm.

[5] W. Eddy, biên tập (2022). *Transmission Control Protocol (TCP)*. RFC 9293. <https://www.rfc-editor.org/rfc/rfc9293.html>. Dùng cho kết nối TCP, luồng byte có thứ tự và cơ chế tin cậy.

[6] R. Fielding, M. Nottingham và J. Reschke, biên tập (2022). *HTTP Semantics*. RFC 9110. <https://www.rfc-editor.org/rfc/rfc9110.html>. Dùng cho HTTP, HTTPS, yêu cầu, phản hồi và trạng thái HTTP. Định nghĩa thời gian từng trường của collector lấy từ mã nguồn, không gán cho RFC này.

- [7] F. T. Liu, K. M. Ting và Z.-H. Zhou (2008). *Isolation Forest*. 2008 IEEE International Conference on Data Mining, tr. 413-422. DOI: 10.1109/ICDM.2008.17. Bản của tác giả: <https://cs.nju.edu.cn/zhoush/zhoush.files/publication/icdm08b.pdf>. Dùng cho ý tưởng cô lập điểm khác thường bằng cây ngẫu nhiên.
- [8] scikit-learn developers. *IsolationForest API reference*. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>. Dùng cho `predict`, `decision_function`, `score_samples`, `contamination` và `random_state`.
- [9] pandas developers. *pandas.DataFrame.rolling*. <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.rolling.html>. Dùng cho cửa sổ theo số quan sát, cửa sổ theo thời gian, `min_periods` và quy tắc biên cửa sổ.
- [10] scikit-learn developers. *Common pitfalls and recommended practices*, phần Data leakage. https://scikit-learn.org/stable/common_pitfalls.html. Dùng cho tách dữ liệu trước khi học các bước tiền xử lý và chỉ fit trên tập huấn luyện.

PHỤ LỤC

A. Tuyên bố sử dụng công cụ hỗ trợ

Nhóm em sử dụng công cụ AI hỗ trợ rà soát yêu cầu, đọc mã thu thập, xây dựng và kiểm tra pipeline. Công cụ không được xem là nguồn xác nhận tính nguyên trạng của dữ liệu hoặc thay thế các RFC và tài liệu thư viện đã dẫn. Nhóm chịu trách nhiệm đọc, kiểm tra, chỉnh sửa và giải thích nội dung trước khi nộp.

B. Đối chiếu yêu cầu của đề tài

Yêu cầu	Sản phẩm
Bài toán mạng và kiến thức liên quan	Chương 1 và Chương 2
Dataset, schema, sampling và làm sạch	Chương 3, CSV, bảng kiểm tra thiếu
Code và khả năng tái lập	Chương 4, phan_tich.py, requirements.txt, README
Một mô hình ML và baseline	Isolation Forest chính; quy tắc ngưỡng đối chiếu
Timeline, khoảng bất thường, chỉ số tăng trội	Hình 1, Bảng 8, intervals.csv
Quan hệ RTT DNS HTTP	Mục 5.4 và Hình 2
Ít nhất ba biểu đồ có ý nghĩa	Bảy hình với diễn giải trong chương kết quả và phụ lục
Đánh giá và giới hạn đúng loại bài toán	Anomaly score, phân tích khoảng, độ nhạy, không dùng nhãn giả định

Demo do nhóm thực hiện	HUONG_DAN_DEMO.md
-------------------------------	-------------------

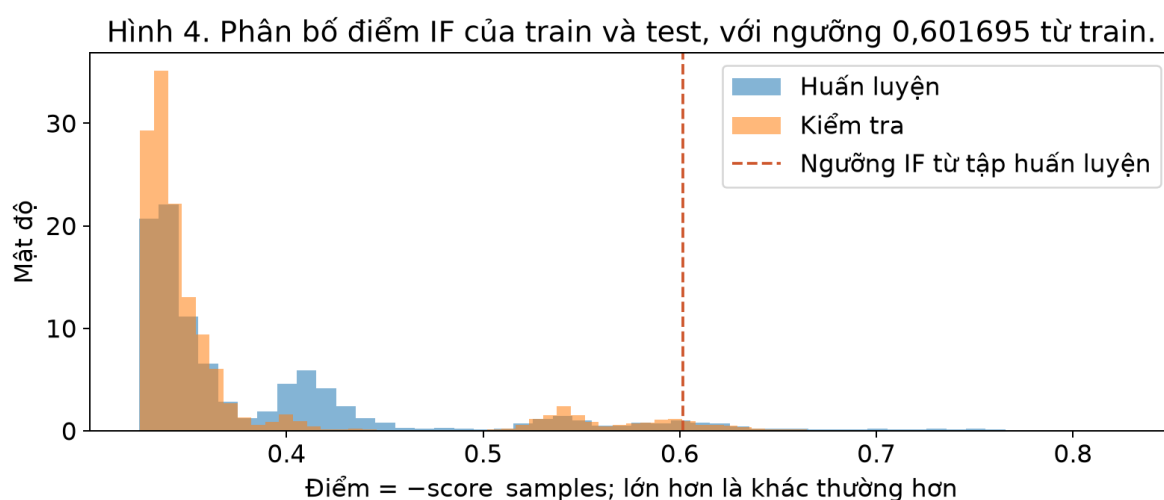
C. Độ nhạy của ngưỡng Isolation Forest

Bảng 10. Đổi ngưỡng từ điểm train của cùng rừng đã huấn luyện

Contamination	Ngưỡng điểm	Cảnh báo test	Tỷ lệ test
2%	0,664682	16	0,10%
5%	0,601695	402	2,63%
10%	0,541167	1329	8,68%

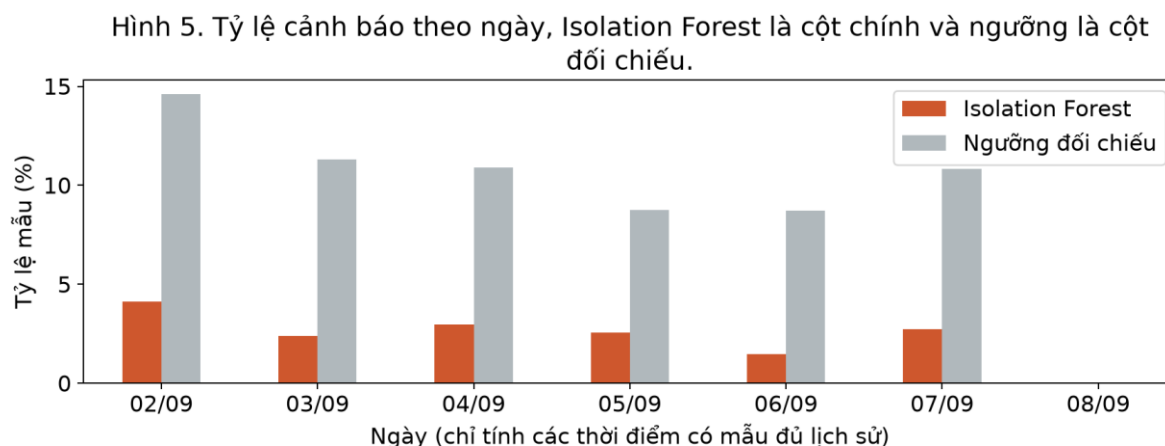
Tăng contamination từ 2% lên 10% làm số cảnh báo test tăng từ 16 lên 1.329. Đây là thay đổi tiêu chuẩn chọn mẫu khác thường, không tự chứng minh phát hiện tốt hơn. Cấu hình chính vẫn là 5%; nếu cần chọn tham số có hệ thống, dùng một tập validation riêng rồi giữ test cho đánh giá cuối.

D. Các biểu đồ bổ sung



Hình 4. Phân bố điểm IF của train và test, với ngưỡng 0,601695 từ train.

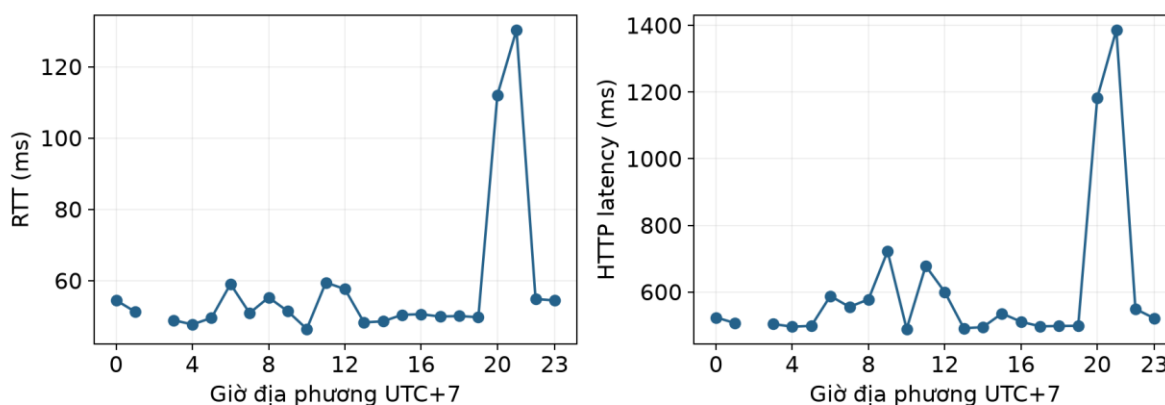
Điểm cao hơn biểu thị khác thường hơn. Tỷ lệ cảnh báo test 2,63% không bị ép bằng contamination 5%. Thay dữ liệu train có thể thay đổi rừng và nhãn dù giữ cùng seed; cần chạy lại toàn bộ khi đổi phiên bản CSV.



Hình 5. Tỷ lệ cảnh báo theo ngày, Isolation Forest là cột chính và ngưỡng là cột đối chiếu.

Biểu đồ dùng số mẫu đủ lịch sử trong mỗi ngày làm mẫu số. Nhóm không suy ra độ phủ 24 giờ từ ngày đầu/cuối hoặc giờ bị thiếu. Cột ngưỡng cao hơn cho thấy cách chọn rộng hơn, chưa thể gọi đó là nhiều báo động sai hơn.

Hình 6. Trung bình RTT và HTTP theo những giờ có dữ liệu.



Hình 6. Trung bình RTT và HTTP theo những giờ có dữ liệu.

Giờ 02 không có mẫu do lịch nghỉ đo; trung bình giờ 01 chỉ phản ánh các mẫu đúng 01:00, không đại diện cả giờ. Nhóm không suy rộng các đợt tăng thành giờ cao điểm cố định của mọi mạng. Cần xem thêm timeline và số lượt quan sát của từng giờ.

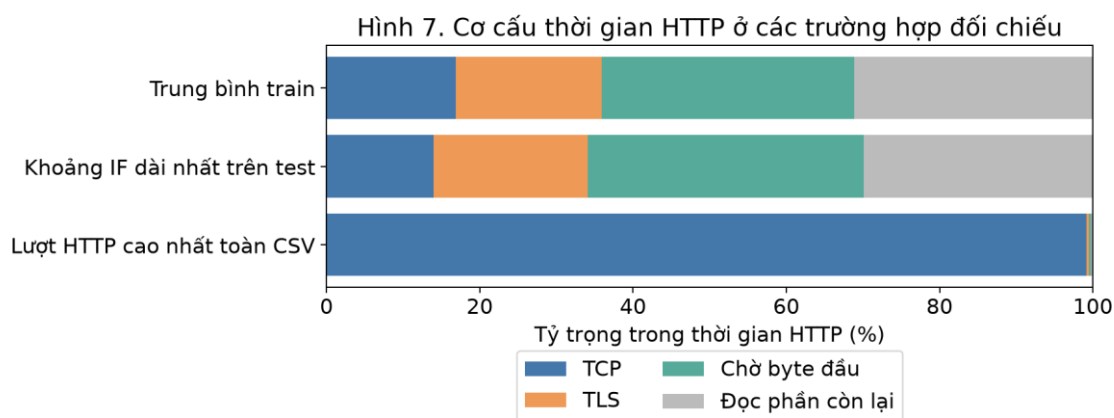
E. Thống kê mô tả toàn bộ CSV

Bảng 11. Thống kê các giá trị hợp lệ trước khi tạo cửa sổ

Chỉ số	Mean	Median	P95	Max
RTT ms	58,43	49,93	128,66	1.848,00
Jitter ms	6,08	0,99	15,11	3.088,67
Loss %	1,01	0,00	0,00	100,00
DNS ms	50,17	45,42	131,93	4.945,65
HTTP ms	610,98	497,90	1.357,11	47.294,06

Mean là trung bình, Median là trung vị, P95 là phân vị 95% và Max là giá trị cao nhất. HTTP có trung bình 610,98 ms nhưng trung vị 497,90 ms; một số giá trị cao làm trung bình tăng. Loss có P95 bằng 0 vì ít nhất 95% lượt có giá trị 0, dù vẫn xuất hiện những lượt không nhận phản hồi. Nhóm không lấy một cực trị để mô tả chất lượng của toàn bộ giai đoạn.

F. Cơ cấu thời gian HTTP



Hình 7. Tỷ trọng TCP, TLS, TTFB và phần đọc còn lại trong ba trường hợp.

Ở mức tham chiếu train, nhóm tính tỷ trọng từ các trung bình thành phần chia cho trung bình HTTP tổng. Cách tính tương tự được dùng cho khoảng IF; thanh cuối dùng một lượt HTTP cực đại. Hình so sánh cơ cấu, không so sánh độ lớn thời gian tuyệt đối. Trường hợp cực đại tập trung ở TCP; khoảng IF ngày 02/09 tăng ở nhiều bước, với TTFB đóng góp lớn nhất vào mức tăng tuyệt đối như Bảng 9.